

Compte-rendu hebdomadaire

Travaux d'Étude et de Recherche

Semaine 2 — Du 6 au 10 avril 2026

SUJET Recommandation

mangas & jeux vidéo

ENCADRANT JACQUENET François

ÉTUDIANT ALI ASSOUMANE Mtara

DATE 10 avril 2026

Table des matières

1	Résumé de la semaine	2
2	Travail réalisé	2
2.1	Conception de l'architecture technique	2
2.1.1	Architecture globale	2
2.1.2	Schéma de la base de données	2
2.1.3	API REST — endpoints définis	3
2.2	Création de la base de données	3
2.3	Implémentation du backend Spring Boot	3
2.3.1	Environnement et dépendances	3
2.3.2	Modèle de données et accès à la base	4
2.3.3	Sécurité et authentification	4
2.3.4	Points d'entrée de l'API	4
2.3.5	Tests des endpoints	4
3	État d'avancement	5
4	Objectifs de la semaine prochaine	6
5	Conclusion	6

1 Résumé de la semaine

Cette deuxième semaine a marqué le passage de la phase de réflexion à la phase de réalisation. Conformément aux objectifs fixés en semaine 1, les travaux ont porté sur trois axes : la conception détaillée de l'architecture technique du système, la création et la configuration de la base de données MySQL, et l'implémentation du backend Spring Boot avec authentification JWT. Une part significative du temps a également été consacrée à la résolution de problèmes techniques rencontrés lors de la mise en place de l'environnement de développement.

2 Travail réalisé

2.1 Conception de l'architecture technique

Avant toute implémentation, une réflexion approfondie sur l'architecture globale du système a été conduite afin de définir les interfaces entre les composantes et d'anticiper les contraintes techniques.

2.1.1 Architecture globale

Le système s'articule en trois couches distinctes et communicantes : une couche mobile Android (Kotlin), une couche backend Spring Boot exposant une API REST sécurisée par JWT, et un moteur de recommandation Python qui sera exposé via une API Flask interne. Cette séparation des responsabilités facilite la maintenance et l'évolution indépendante de chaque composante.

2.1.2 Schéma de la base de données

La modélisation de la base de données a abouti à un schéma de 7 tables interconnectées. Un choix de conception important a été de regrouper les mangas et les jeux vidéo dans une même table `items`, différenciés par un champ `type` (ENUM manga/game). Cette approche simplifie les requêtes de recommandation cross-domaine, qui constituent le cœur du projet.

Table	Rôle
<code>users</code>	Comptes utilisateurs (UUID, email, mot de passe hashé BCrypt)
<code>items</code>	Oeuvres unifiées mangas et jeux (type ENUM manga/game)
<code>genres</code>	Référentiel des genres
<code>item_genre</code>	Liaison entre les œuvres et leurs genres (une oeuvre peut avoir plusieurs genres)
<code>ratings</code>	Notes utilisateurs (score 0–10, contrainte unicité user/item)
<code>history</code>	Historique de consultation (signal implicite pour le moteur IA)
<code>recommendations</code>	Recommandations générées (score, méthode : collaborative/content/hybrid)

2.1.3 API REST — endpoints définis

Les endpoints ont été définis, répartis en 6 groupes fonctionnels : authentification, catalogue, notations, historique, recommandations et gestion du profil utilisateur.

2.2 Création de la base de données

La base de données MySQL `ter_reco` a été créée et initialisée via un script SQL. Les 7 tables ont été créées avec leurs contraintes (clés primaires, clés étrangères avec cascade, contraintes d'unicité).

2.3 Implémentation du backend Spring Boot

2.3.1 Environnement et dépendances

Le projet Spring Boot a été initialisé sous VSCode avec les dépendances suivantes : Spring Web, Spring Data JPA, Spring Security, MySQL Driver, Lombok et la bibliothèque `jjwt` pour la gestion des tokens JWT. La mise en place de l'environnement a nécessité plusieurs ajustements et résolutions de problèmes, notamment une incompatibilité entre le type `CHAR(36)` de MySQL et le type `VARCHAR` attendu par Hibernate (résolue en passant `ddl-auto` à `none`), ainsi que des problèmes de traitement des annotations Lombok sous VSCode (résolus en remplaçant les annotations par des constructeurs explicites).

2.3.2 Modèle de données et accès à la base

Chaque table de la base de données est représentée dans le code Java par une classe dédiée : utilisateur, œuvre, genre, note, historique et recommandation. Ces classes permettent à Spring Boot de lire et d'écrire en base de données sans écrire de requêtes SQL manuellement. Des composants de recherche ont également été mis en place pour permettre, par exemple, de retrouver toutes les notes d'un utilisateur ou de filtrer le catalogue par type d'œuvre (manga ou jeu vidéo).

2.3.3 Sécurité et authentification

La sécurité de l'API repose sur un système de tokens JWT (JSON Web Token). Lors de la connexion, le serveur génère un token unique valable 24 heures, que le client devra joindre à chacune de ses requêtes pour prouver son identité. Toute requête sans token valide est automatiquement rejetée. Les mots de passe ne sont jamais stockés en clair — ils sont transformés par un algorithme de hachage (BCrypt) avant d'être enregistrés en base de données.

2.3.4 Points d'entrée de l'API

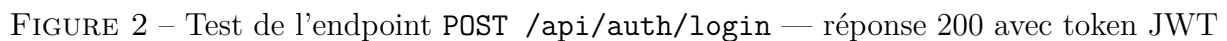
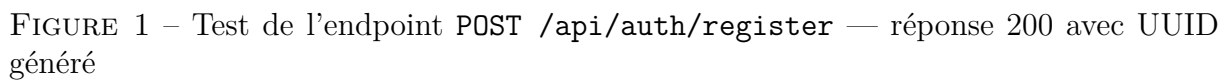
L'API a 6 groupes de fonctionnalités qui sont pour l'application mobile Android dont le développement n'a pas encore débuté. La gestion des comptes (inscription et connexion), la consultation du catalogue d'œuvres (avec recherche et filtres), la notation des mangas et jeux vidéo, l'enregistrement de l'historique de consultation, la récupération des recommandations personnalisées, et la gestion du profil utilisateur. Ces endpoints sont d'ores et déjà opérationnels et ont été testés manuellement via Thunder Client en attendant l'intégration mobile.

2.3.5 Tests des endpoints

Les endpoints d'authentification ont été testés avec succès via Thunder Client (extension VSCode) :

Endpoint	Résultat	Statut
POST /api/auth/register	Compte créé, UUID retourné	200 OK
POST /api/auth/login	Token JWT retourné	200 OK

Les captures ci-dessous illustrent les réponses obtenues lors des tests :



3 État d'avancement

10 Avril 2026 Page 5 sur 6 TER M1 DSC

4 Objectifs de la semaine prochaine

- La mise en place de l'environnement Python et l'implémentation du moteur de recommandation : filtrage collaboratif , filtrage basé sur le contenu et moteur hybride
- L'alimentation de la base de données avec des données réelles via l'API Jikan (MyAnimeList) pour les mangas

5 Conclusion

Cette deuxième semaine a permis de poser les fondations techniques du projet. L'architecture a été conçue en détail, la base de données est opérationnelle avec ses 7 tables, et le backend Spring Boot expose une API REST sécurisée par JWT, testée avec succès.